

计算机组成与体系结构

第16章 指令级并行与超标量处理器

Chapter 16 Instruction Level Parallelism
and Superscalar Processors

William Stallings, 10th Edition

用途	考前复习资料 / Exam Review Cheat Sheet
范围	16.1 Overview ~ 16.2 Design Issues (至Branch Prediction之前)
语言	中文为主，关键术语附英文
页码引用	每条要点标注来源幻灯片页码
生成日期	2026-06-03

一、超标量概述 / Superscalar Overview (p. 1 – 10)

1.1 什么是超标量？

- 定义 (Superscalar)

超标量 (Superscalar) 一词于1987年首次提出，指一种通过执行标量指令来提升性能的机器设计方法。其核心是让多条指令在不同流水线中独立、并发地执行，且允许指令以不同于程序顺序的次序执行。(p. 6)

- 关键能力 (Key Capabilities)
- 常见指令 (算术、加载/存储、条件分支) 可被同时发起并独立执行 (p. 5)
- 对 RISC 和 CISC 同样适用 (p. 5)
- 已成为高性能微处理器的标准方法 (p. 5)

1.2 超标量 vs 超级流水线 (Superscalar vs Superpipeline) (p. 7 – 10)

- 超级流水线 (Superpipeline)
- 许多流水级用时少于半个时钟周期；双倍内部时钟速度，每个外部时钟周期完成两个任务 (p. 9)
- 超标量 (Superscalar)

-

允许并行取指和执行。在程序开始和每个分支目标处，超级流水线处理器落后于超标量处理器 (p. 10)

- 标量 (Scalar)
- 单一流水线功能单元，允许多条指令同时处于不同流水级 (p. 7)
- 超标量 (Superscalar)
- 多个功能单元，每个功能单元本身也是流水线实现的，每个单元提供一定的并行度 (p. 8)

二、指令级并行及其限制 / ILP and Its Limitations (p.11 - 17)

2.1 指令级并行 (ILP)

- 指令级并行 (Instruction Level Parallelism — ILP)
- 程序中的指令在平均意义上能够被并行执行的程度 (p.11)
- 由代码中真数据依赖和过程依赖的频率决定 (p.19)
- 还受操作延迟 (Operation Latency) 支配：指令结果可用作后续指令操作数所需的时间 (p.19)

2.2 提高 ILP 的方法

- 编译器优化 (Compiler-based Optimization)
- 硬件技术 (Hardware Techniques)

2.3 五大限制因素 (p.11, p.45)

限制类型	英文名称	别名	说明	页码
真数据依赖	True Data Dependency	RAW (Read-After-Write)	后一条指令使用前一条指令产生的结果，无法并行执行	p.11, p.45
过程依赖	Procedural Dependency	分支依赖	条件分支后的指令不能与分支前的指令并行执行；CISC中可变量指	p.11, p.45
资源冲突	Resource Conflict	结构冲突	两条以上指令同时需要同一资源 (如ALU)，可通过复制资源解	p.11, p.45
输出依赖	Output Dependency	WAW (Write-After-Write)	后一条指令先于前一条指令写回结果，导致寄存器值错误	p.11, p.45
反依赖	Antidependency	WAR (Write-After-Read)	后一条指令先于前一条指令读取之前写回，破坏了源操作数值	p.11, p.45

2.4 依赖详解

- 真数据依赖 / RAW (True Data Dependency / Flow Dependency)

```
ADD EAX, ECX ; MOV EBX, EAX ——
```

第二条指令可以与第一条并行取指和译码，但不能在第一条完成之前执行。(p.12)

- 涉及内存访问时延迟更长 (LOAD/STORE可能需2个以上时钟周期) (p.13)
- 对RISC：重排序指令可加速流水线；对超标量：重排序的效果不如RISC流水线显著 (p.13)
- 过程依赖 (Procedural Dependency)
- 条件分支后的指令不能与分支前的指令并行执行 (p.14)
- CISC中指令长度不固定，需先译码确定取指次数——阻止同时取指，这是超标量更适用于RISC的原因之一 (p.14)
- 输出依赖与反依赖 (Output & Anti Dependency)
- 两者不同于真数据依赖和资源冲突，都属于存储冲突 (Storage Conflicts) 的实例 (p.34)
- 本质原因：寄存器内容可能无法反映程序正确的执行顺序 (p.35)

三、设计问题 / Design Issues (p. 18 - 41)

3.1 指令级并行 vs 机器并行

- 机器并行 (Machine Parallelism)
- CPU 利用指令级并行的能力 (p. 21)
- 受限于：(1) 寻找独立指令的机制的速度和复杂度；(2) 可并行取指和执行的流水线数量 (p. 21)

3.2 指令发射策略 (Instruction Issue Policy) (p. 22 - 33)

- 指令发射 (Instruction Issue)
- 在处理器功能单元中发起指令执行的过程 (p. 22)
- 指令发射策略 (Instruction Issue Policy)
- 用于发射指令的协议。CPU必须「向前看」 (look ahead) 以寻找可并行执行的指令 (p. 22)
- 三种重要顺序 (Three Important Orders)
- (1) 取指顺序 (Fetch Order)
- (2) 执行顺序 (Execution Order)
- (3) 写回 (更改寄存器和内存) 顺序 (Write Order) (p. 22)

策略	取指/译码	执行	写回	效率	停顿原因	页码
按序发射 按序完成 (In-Order Issue In-Order Completion)	按序	按序	按序	效率不高	资源冲突、真数据依赖、过程依赖、功能单元需要多周期	
按序发射 乱序完成 (In-Order Issue Out-of-Order Completion)	按序	乱序	乱序	中等	资源冲突、真数据依赖、过程依赖 (执行阶段空闲即可执行)	p. 28 - 29
乱序发射 乱序完成 (Out-of-Order Issue Out-of-Order Completion)	乱序	乱序	乱序	最高	使用指令窗口解耦译码和执行， 降低流水级停顿概率	p. 30 - 33

3.3 指令窗口 (Instruction Window) (p. 31 - 33)

- 指令窗口 (Instruction Window)
- 解耦译码流水线和执行流水线的关键机制 (p. 31)
- 允许持续取指和译码直到缓冲区满 (p. 32)
- 当功能单元可用时，窗口中无冲突/无依赖的指令即可执行 (p. 32)
- 不是一个额外的流水段 (p. 32)
- 使处理器能向前看 (look ahead)，从窗口中识别独立指令 (p. 32)
- 乱序发射减少了流水级停顿的概率 (p. 33)

四、寄存器重命名 / Register Renaming (p. 34 – 37)

4.1 为什么需要寄存器重命名？

- 输出依赖和反依赖本质上是存储冲突 (Storage Conflicts)，原因是寄存器内容可能无法反映正确的程序顺序 (p. 35)
- 可能导致流水线停顿 (p. 35)
- 编译器对寄存器的优化 (第15章中讨论) 可能使冲突更加尖锐 (p. 35)

4.2 寄存器重命名机制

- 寄存器重命名 (Register Renaming)
- 本质是资源的复制 (Duplication of Resources) (p. 35)
- 由处理器硬件动态分配寄存器 (p. 35)
- 使用下标区分逻辑寄存器 (无下标) 和硬件分配的物理寄存器 (有下标)，减少不必要的输出/反依赖 (p. 36)

- 重命名示例 (p. 36 – 37)：

I1: R3b $\leftarrow$ R3a op R5a (真数据依赖: I1 $\rightarrow$ I2)

I2: R4b $\leftarrow$ R3b + 1 (真数据依赖: I2 $\rightarrow$ I4)

I3: R3c $\leftarrow$ R5a + 1 (真数据依赖: I3 $\rightarrow$ I4)

I4: R7b $\leftarrow$ R3c op R4b

重命名后消除了 I1-I3 (WAW) 和 I2-I3 (WAR) 的伪依赖。

4.3 机器并行性评估 (p. 38 – 39)

- 三种提升性能的硬件技术 (p. 38)：
- (1) 复制资源 (Duplication of Resources)：加载/存储/ALU等功能单元
- (2) 乱序发射 (Out-of-Order Issue)：指令窗口
- (3) 寄存器重命名 (Register Renaming)：重复寄存器
- 仿真结论 (Simulation Conclusions) (p. 38 – 39)：
- 没有寄存器重命名的情况下，增加功能单元是不值得的 (Not worthwhile to add function units without register renaming)
- 需要足够大的指令窗口 (more than 8) (Need instruction window large enough)

五、分支预测 / Branch Prediction (p. 40 - 41)

- 高性能流水线机器必须处理分支问题 (p. 40)
- 80486 (CISC)：同时取分支后下一条顺序指令和分支目标指令 (p. 40)
- RISC 机器使用延迟分支 (Delayed Branch) 策略 (p. 40)：
- 在不可用指令被预取之前计算分支结果 (p. 40)
- 始终执行分支后的单条指令，在取新指令流的同时保持流水线充满 (p. 40)
- 延迟分支对超标量的局限性 (Delayed Branch Limitations for Superscalar)
- 延迟分支策略对超标量机器吸引力较小 (p. 41)
- 原因：(1) 延迟槽中需执行多条指令；(2) 指令依赖问题 (p. 41)
- 分支预测技术回归 (Return of Branch Prediction)
- 静态分支预测 (Static Branch Prediction)：PowerPC 601 (p. 41)
- 动态分支预测 (Dynamic Branch Prediction)：PowerPC 620 和 Pentium 4 (p. 41)

六、超标量执行与实现 / Superscalar Execution & Implementation (p. 42 – 43)

6.1 超标量执行流程 (p. 42, Fig 16.7)

- 顺序取指 (Sequential Fetch) → 顺序译码 (Sequential Inst. Decode) → 按序重排 (In-order Reorder) → 乱序执行 (Out-of-order Execute)
- 基于数据依赖和资源冲突来决定执行顺序 (p. 42)

6.2 超标量实现关键要素 (p. 43)

- (1) 同时取多条指令 (Simultaneously fetch multiple instructions)
- (2) 判断寄存器值的真依赖的逻辑, 以及传递这些值的机制
- (3) 并行发起多条指令的机制
- (4) 并行执行多条指令的资源
- (5) 按正确顺序提交 (commit) 处理状态的机制

七、关键术语速查表 / Key Terminology (全章)

中文术语	English Term	缩写	页码
超标量	Superscalar		p. 5-6
超级流水线	Superpipeline		p. 9-10
指令级并行	Instruction Level Parallelism	ILP	p. 11, 19
机器并行	Machine Parallelism		p. 21
操作延迟	Operation Latency		p. 19
真数据依赖	True Data Dependency / Flow Dependency	RAW	p. 11-13
过程依赖	Procedural Dependency		p. 14
资源冲突	Resource Conflict		p. 15
输出依赖	Output Dependency	WAW	p. 17, 34
反依赖	Antidependency	WAR	p. 17, 34
存储冲突	Storage Conflict		p. 34
指令发射	Instruction Issue		p. 22
指令发射策略	Instruction Issue Policy		p. 22
按序发射	In-Order Issue		p. 23
乱序完成	Out-of-Order Completion		p. 28
乱序发射	Out-of-Order Issue		p. 30-31
指令窗口	Instruction Window		p. 31-33
寄存器重命名	Register Renaming		p. 35-37
延迟分支	Delayed Branch		p. 40-41

中文术语	English Term	缩写	页码
分支预测	Branch Prediction		p. 40-41
静态分支预测	Static Branch Prediction		p. 41
动态分支预测	Dynamic Branch Prediction		p. 41

八、核心公式与速记 / Key Formulas & Memory Hooks

- $ILP \text{ 程度} \propto 1 / (\text{真数据依赖频率} + \text{过程依赖频率})$ —— 依赖越多，并行度越低 (p. 19)
- 机器并行能力上限 = $\min(\text{寻找独立指令的机制能力}, \text{可并行执行的功能单元数量})$ (p. 21)
- 寄存器重命名 = 资源复制 + 动态分配 ——
消除伪依赖 (WAR/WAW) 但不消除真依赖 (RAW) (p. 35)
- 三大硬件优化: 复制资源 $\rightarrow$ 乱序发射 $\rightarrow$ 寄存器重命名 (按效果递增) (p. 38)
- 经验法则: 无寄存器重命名 = 加功能单元无意义; 指令窗口大小 > 8 才有效 (p. 38-39)

九、典型例题提示 / Problem-Solving Hints

- 例题类型 (Typical Exam Questions)
 - (1) 给定指令序列，识别依赖类型 (RAW/WAW/WAR) (参考p. 17, 34示例)
 - (2) 给定指令序列和功能单元配置，画出不同发射策略下的时空图 (参考p. 24-27按序完成、p. 29乱序完成、p. 33乱序发射的执行图)
 - (3) 对给定代码段进行寄存器重命名，消除伪依赖 (参考p. 36-37示例)
 - (4) 比较超标量与超级流水线的性能差异 (参考p. 9-10)
- 易错点 (Common Pitfalls)
 - 混淆 RAW 和 WAW: RAW 是真依赖 (不能消除)，WAW/WAR 是伪依赖 (可通过寄存器重命名消除)
 - 指令窗口不是一个流水段——它是一个缓冲区，用于解耦译码和执行
 - 乱序执行 $\neq$ 结果乱序 —— 必须按正确顺序提交 (commit) 过程状态
 - 延迟分支在超标量中效果有限，因为延迟槽需要填充多条指令

— 祝考试顺利 —